Project: Service Mesh & Observability

Due: 04/18/2026 11:59 PM
Extended Deadline: 04/19/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Istio
- Task Overview
- Task

Prometheus
- Task Overview
- Task

Open Telemetry
- Task Overview
- Task

Wrap Up
- Conclusion

Istio (Task)

Task 1 - Hands off Networking with Istio

Assumptions

This task makes the following assumptions:

1. You have your cluster running with 3 nodes
2. Tekton and FluxCD are setup in your cluster and that Tekton builds and pushes your images
3. The following keys are setup in your cluster:
   - Docker Credentials
   - Git Credentials
   - Login Keys
4. It's okay if your login and llm pods are not running

Do NOT proceed with this task if you don't meet the above assumptions. For the benefit of doubt, your cluster state should look like the following:

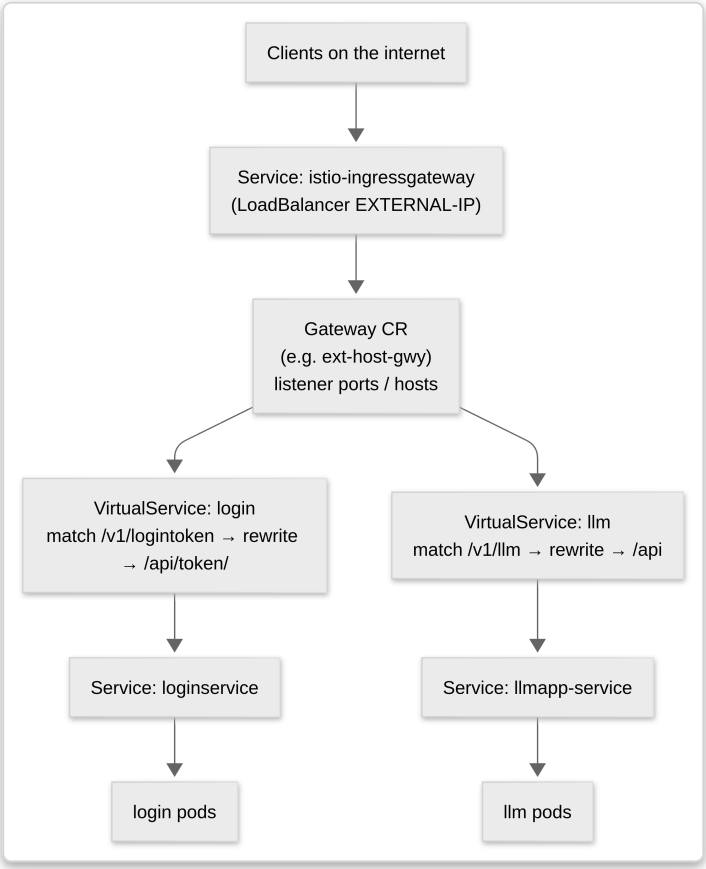
```
azureuser@qa-vm:~/cloud-native-handout-mil
NAME            REVISION                SU
cdpipeline      master@sha1:8e7f4e4d    Fa
cipipeline      master@sha1:8e7f4e4d    Fa
crds            master@sha1:8e7f4e4d    Fa
flux-system     master@sha1:8e7f4e4d    Fa
helm-releases   master@sha1:8e7f4e4d    Fa
helm-sources    master@sha1:8e7f4e4d    Fa

azureuser@qa-vm:~/cloud-native-handout-mil
NAME
azure-storage-account-fdf89ebc9257d4c1c9f8
docker-credentials
git-credentials
loginkeys
```

Remember the doorman from Unit 3?

We introduced the Gateway API in Unit 3 and used NGINX to power it, with KrakenD validating login tokens. In this project, we're upgrading to a more powerful gateway technology: **Istio**.

Task 1: Implementing a Service Mesh with Istio

We're integrating Istio into our Kubernetes cluster to level up its networking, security, and visibility. Here's what that means and why it matters:



What is Istio?

Istio is a **service mesh** — a dedicated infrastructure layer that manages how services communicate with each other inside your cluster. Instead of baking networking logic into each service, Istio handles it transparently.

Why are we switching to Istio?

1. Advanced Traffic Management

Istio gives you fine-grained control over how traffic flows between services — all configured through YAML files you already know. Think intelligent routing, retries,

timeouts, and canary deployments without touching application code.

2. Built-in Security
Istio enables **mutual TLS (mTLS)** automatically, meaning every service-to-service connection inside the cluster is encrypted and authenticated — with zero changes to your app code.

3. Observability
Istio comes with built-in dashboards for monitoring traffic, tracing requests across services, and spotting bottlenecks — giving you visibility that was previously difficult to achieve in a microservices environment.

> **Key shift from Unit 3:** NGINX + KrakenD handled gateway concerns at the *edge* of your cluster. Istio operates *inside* the cluster, managing every hop between your microservices — not just the front door.

Subtask 1 - Add Istio Helm Charts

With a fully built CI/CD pipeline in place, installing new resources is straightforward. To install Istio, you'll need to locate and apply its associated HelmRepository and HelmRelease manifests — these tell Flux where to pull the charts from and how to install them.

We've provided these manifests for you as a starting point. Use them as a reference alongside the official Flux documentation to understand how they work and get Istio running in your cluster.

> Flux HelmRepository API:
> https://fluxcd.io/flux/components/source/helmrepositories/
> Flux HelmRelease API:
> https://fluxcd.io/flux/components/helm/helmreleases/

Download the handout for task1

You will use the following starter code for task 1. Please download this file outside your repository.

```
curl https://cloudnativecourse.blob.core.w
```

Your handout should have the following tree structure:

```
├── istio-releases
│   ├── istio-base.yaml
│   ├── istio-discovery.yaml
│   └── istio-ingress-gateway.yaml
├── istio-sources
│   ├── istio-namespace.yaml
│   └── istio-repo.yaml
└── k8s
    └── istio
        ├── gateway.yaml
        ├── kustomization.yaml
        ├── namespace.yaml
        ├── security.yaml
        └── virtual-service.yaml
```

Let's first focus on istio-sources and istio-releases. Here's a breakdown:

`istio-sources/` — Sets up the prerequisites before anything can be installed.

- `istio-namespace.yaml` — Creates the `istio-system` namespace where all Istio components will live.
- `istio-repo.yaml` — Registers the official Istio Helm chart repository so Flux knows where to pull charts from.

`istio-releases/` — Installs the three core components of Istio, in order.

- `istio-base.yaml` — Installs the cluster-wide CRDs and base resources that all other Istio components depend on.
- `istio-discovery.yaml` — Installs Istiod, the Istio control plane. This is the brain of the service mesh — it handles config, service discovery, and certificate management.
- `istio-ingress-gateway.yaml` — Installs the ingress gateway, which is the entry point for all external traffic into the cluster.

Information

Before proceeding, go through the `istio-repo.yaml` and all files in the istio-releases folder to get an overall picture of what they're doing.

Your Tasks

1. Copy the contents of istio-sources to /helm-sources
2. Copy the contents of istio-releases to /helm-releases
3. Copy the entire `k8s` folder into your project's root directory
4. Update the `kustomization.yaml` files in **helm-sources** and **helm-releases** to include the newly added resources.
5. Remove mentions of **nginx-gateway** from the both `kustomization.yaml` files in **helm-sources** and **helm-releases**. Since Istio is now acting as our Gateway, we no longer need NGINX.

Commit, Push & Reconcile Flux

Whenever changes such as this are made, you should commit, push your changes and immediately reconcile Flux.
We always want to make sure that changes we're making aren't breaking stuff.

Bookmark the following commands. You'll use them throughout this task:

```
flux reconcile source git flux-system

flux get kustomizations

kubectl get svc -n istio-system
```

Interim Validation - Cluster State

After making the above changes, your cluster state should look like this:

```
azureuser@qa-vm:~/cloud-native-handout-mil
NAME            REVISION                SU
cdpipeline      master@sha1:8e7f4e4d    Fa
cipipeline      master@sha1:8e7f4e4d    Fa
crds            master@sha1:8e7f4e4d    Fa
flux-system     master@sha1:8e7f4e4d    Fa
helm-releases   master@sha1:8e7f4e4d    Fa
helm-sources    master@sha1:8e7f4e4d    Fa

azureuser@qa-vm:~/cloud-native-handout-mil
NAME                    TYPE           CLU
```

```
istio-ingressgateway   LoadBalancer   10.(
istiod                  ClusterIP      10.
```

Take note of the `istio-ingressgateway` and `istiod` resources that are newly added.

Subtask 2 - Add Kubernetes Manifests to the k8s Directory

In this subtask, you will leverage FluxCD to deploy your Istio gateway resources, your LLM and Login service.

1. Clean up existing resources using Kubectl delete

Delete all resources in `django-authentication/k8s/` by running kubectl delete .....

Delete all resources in `llm_server/k8s/` by running kubectl delete .....

2. Create new directories inside `k8s/`

```
k8s/llm
k8s/login
```

3. Migrate existing manifests

Copy all files from `django-authentication/k8s/` to `k8s/login/`

Copy all files from `llm_server/k8s/` to `k8s/llm/`

4. Add a `kustomization.yaml` to each new directory

`k8s/login/kustomization.yaml` — resources in this order:

```
resources:
  - deployment.yaml
  - service.yaml
```

`k8s/llm/kustomization.yaml` — resources in this order:

```
resources:
  - model-pvc.yaml
```

```
  - deployment.yaml
  - service.yaml
```

Set the Default Namespace

Flux does not automatically assign namespaces to resources, so you need to explicitly add `namespace: default` to the `metadata` block of every resource in the `k8s/login` and `k8s/llm` folders.

5. Verify your directory structure before proceeding

```
k8s
├── istio
│   ├── gateway.yaml
│   ├── kustomization.yaml
│   ├── namespace.yaml
│   └── security.yaml
├── llm
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   ├── model-pvc.yaml
│   └── service.yaml
└── login
    ├── deployment.yaml
    ├── kustomization.yaml
    └── service.yaml
```

You can safely delete the k8s folders in the **llm_server** and **django-authentication** folders since the process is now automated with Flux. With GitOps in place, we no longer need these folders.

6. Update the Kustomize Sources

Flux uses the `my-cluster/` directory to know which folders in your repo to watch for Kubernetes manifest changes. You need to add the following three files to `my-cluster/kustomize-sources/`:

```
istio.yaml
llm.yaml
login.yaml
```

The `istio.yaml` has been provided for you as a reference:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: istio
  namespace: flux-system
spec:
  interval: 5m
  path: ./k8s/istio
  prune: true
  dependsOn:
    - name: helm-releases
  sourceRef:
    kind: GitRepository
    name: flux-system
```

Use this as a template for `login.yaml` and `llm.yaml`. Pay close attention to these three fields when adapting it:

Field	Description
`name`	A unique name for this Kustomization
`path`	The folder in your repo Flux should watch
`dependsOn`	What must be successfully deployed before this runs

Before writing your files, think through the following:

Where should `login.yaml` point for its `path`? What does it depend on?
Where should `llm.yaml` point for its `path`? What does it depend on? Should it depend on `login`?

Interim Validation

Commit, push and reconcile Flux.

Your cluster state should look like this:

```
azureuser@qa-vm:~/cloud-native-handout-mil
NAME            REVISION                SU
cdpipeline      master@sha1:e8b74a79    Fa
cipipeline      master@sha1:e8b74a79    Fa
crds            master@sha1:e8b74a79    Fa
flux-system     master@sha1:e8b74a79    Fa
helm-releases   master@sha1:e8b74a79    Fa
```

```
helm-sources    master@sha1:e8b74a79    Fa
istio           master@sha1:e8b74a79    Fa
llm             master@sha1:e8b74a79    Fa
login           master@sha1:e8b74a79    Fa

azureuser@qa-vm:~/cloud-native-handout-mil
NAME                    TYPE           CLUS
istio-ingressgateway    LoadBalancer   10.(
istiod                  ClusterIP      10.(

azureuser@qa-vm:~/cloud-native-handout-mil
NAME                                  READY
llmapp-deployment-7d5dcd7f56-qdfl8    2/2
login-ddf877577-g9tkj                 2/2

azureuser@qa-vm:~/cloud-native-handout-mil
NAME              TYPE           CLUSTER-II
kubernetes        ClusterIP      10.0.0.1
llmapp-service    LoadBalancer   10.0.193.8
loginservice      LoadBalancer   10.0.115.1
```

Why Use Flux for Istio Deployment?

We're using Flux to deploy Istio and its configurations instead of a standalone approach for several reasons:

1. GitOps workflow: Flux allows us to manage Istio configurations as code in our Git repository, ensuring version control and easy rollbacks.
2. Automated synchronization: Flux automatically applies changes to the cluster when configurations are updated in the repository, reducing manual intervention.
3. Consistency: By using Flux, we maintain a consistent deployment approach across our entire infrastructure, including Istio.
4. Declarative configuration: Flux enables us to define the desired state of our Istio setup declaratively, which aligns with Kubernetes and GitOps principles.
5. Integration with existing pipeline: Since we're already using Flux for other aspects of our deployment, incorporating Istio through Flux maintains a unified management approach.

By using Flux to deploy and manage Istio, we ensure that our service mesh configuration remains consistent,

versioned, and automatically updated, aligning with our overall GitOps strategy for the entire Kubernetes cluster.

Subtask3 - VirtualService Configuration

We need to create VirtualService definitions for each of our services. These are similar to HttpRoute definitions in the Gateway API. Add the following starter template to `k8s/virtual-service.yaml`:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: my-service
  namespace: default
spec:
  hosts:
  - "*"
  gateways:
  - ext-host-gwy
  http:
  - match:
    - uri:
        prefix: /api
    route:
    - destination:
        host: my-service
        port:
          number: 80
```

This VirtualService:

1. Metadata

```
name: my-service
namespace: default
```

The VirtualService is named `my-service` and is created in the default namespace.

1. Specification
    1. Hosts

```
hosts:
- "*"
```

```
  This wildcard entry means the VirtualSe
```

1. Gateways

```
gateways:
- ext-host-gwy


  This associates the VirtualService with
```

1. HTTP Routing Rules

```
http:
- match:
  - uri:
      prefix: /api
  route:
  - destination:
      host: my-service
      port:
        number: 80


  This section defines the routing behavi
```

It matches requests with URIs that have the prefix `/api`
Matched requests are routed to: A service named `my-service` On port `80`

Your Turn!

You'll need to create similar VirtualService definitions for each of your services (login, llm.), adjusting the `name`, `uri prefix`, and `destination host` accordingly. By implementing these Istio CRDs, you're setting up a secure and flexible routing system for your microservices architecture. The Gateway defines the entry point for external traffic, while the VirtualServices determine how that traffic is routed to specific services within your cluster.

1. Set up a VirtualService for the LLM service, routing external requests for `/v1/llm` to the internal path `/api`
2. Set up a VirtualService for the Login service, routing external requests for `/v1/logintoken` to the

internal path `/api/token/`

3. The VirtualServices are associated with the Gateway we created earlier

> **Information**
>
> Note that you should put all virtual services in the `virtual-service.yaml` file. You can use **---** each virtual service definition

Before we worry about **authenticating requests** using our **login service**, we first need to **make sure that we can reach our login service**.
You should configure your login **virtualservice** that, **under any host**, exposes the **/v1/logintoken** route, which just maps to the **/api/token/** route for our **loginservice**.
[https://istio.io/latest/docs/reference/config/networking/virtual-service/#HTTPRewrite](This section of the documentation will be helpful.)

Interim Validation

Once this is completed, update your `k8s/istio/kustomization.yaml` file to include the **virtual-service.yaml** resource.
Commit and push your changes to GitHub
Reconcile Flux immediately
Make sure all kustomizations show status: TRUE

You can use `kubectl get svc -n istio-system` to get the IP of you external load balancer.

Once you have done all of this, make sure that you can **get a service token** with:

```
curl   -X POST   -H "Content-Type: applica
```

To get the istio-gateway you this command

```
kubectl get gateways.networking.istio.io
```

Subtask 4 - Securing the Application

Below is a YAML configuration that we'll use to make our service secure. Copy and paste this into the `k8s/istio/security.yaml` file:

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: llm-auth
  namespace: default
spec:
  selector:
    matchLabels:
      app: llmapp
  jwtRules:
  - issuer: "llmlogin"
    jwks: |
      {"keys": [{"kty": "RSA", "n": "YOUR_
---
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: llm-authz
  namespace: default
spec:
  selector:
    matchLabels:
      app: llmapp
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]
```

This configuration does two things:

`RequestAuthentication` — Tells Istio to validate JWTs on any request hitting the `llmapp`. It checks that the token was issued by `llmlogin` (your Django login service) and verifies the token's signature using your public key.

`AuthorizationPolicy` — Tells Istio to block any request that doesn't carry a valid, verified JWT. In other words, unauthenticated users cannot reach the LLM server at all.

Together, these two resources ensure that only users who have logged in and received a valid token from your Django service can access the LLM.

Getting your public key

The `YOUR_PUBLIC_KEY` value is the RSA public key your Django login service uses to sign JWTs.
You used this in the Authentication and API Gateways project. Make sure this key is **base64 encoded**.
If you're generating a new public/private key-pair, make sure to create the Kubernetes loginkeys secret again.

Once you're done with the changes, update the `k8s/istio/kustomization.yaml` file to include the **security.yaml** resource.

Commit and push your changes to GitHub, then run reconcile Flux.

The JWTS Issuer

As you have observed, Istio is expecting that every token generated by Django must have the ISSUER and USER_ID_CLAIM fields.
To do this, you will update the **SIMPLE_JWT** variable your Django `settings.py` file to include:

```
"ISSUER": "llmlogin",
"USER_ID_CLAIM": "sub",
```

Once the above change is implemented, you should:

1. Commit and push your changes
2. Reconcile Flux
3. Once all flux reconcililations are TRUE, you should trigger your pipeline run with the command:

```
kubectl create -f tekton/pipelinerun.yaml
tkn pipeline logs -f -L

tkn pipelinerun describe -L
```

Make sure the `fluxcdbot` has made a push to your deployment files in response to a new image being pushed to the ACR.
Then, run `flux get kustomizations` to see the new image being pushed to the cluster.

Run `git pull` to get the latest code to your local machine.

```
Fast-forward
 k8s/llm/deployment.yaml
```

```
k8s/login/deployment.yaml
```

Interim Validation

Generate Your Istio External Load balancer IP

```
kubectl get svc -n istio-system
```

Generate a new token using

```
export accesstoken=$(curl -X POST -H "Cont
```

Using the generated token, make a request to the llm service:

```
curl -D - -H "Authorization: Bearer $acces
```

Warning

```
content-length: 14
content-type: text/plain
date: Sat, 04 Apr 2026 20:06:45 GMT
server: istio-envoy
x-envoy-upstream-service-time: 7
```

If you get the above error, make sure that your public key is not stale. If it is, you should generate a new key pair, pass store the keys in the cluster again and pass the new public key to the istio authorization policy via the security.yaml file.

Then force flux to reconcile immediately:

```
flux reconcile kustomization istio --with-
```

Submit Your Work

Tree structure

Your directory structure should look like this:

```
├── cdpipeline
│   ├── acrsecret-sealed.yaml
│   ├── image-update-automation.yaml
│   ├── kustomization.yaml
│   ├── llm-image-repository.yaml
```

```
│   ├── llm-policy.yaml
│   ├── login-image-repository.yaml
│   └── login-policy.yaml
├── cipipeline
│   ├── git-clone.yaml
│   ├── kaniko.yaml
│   ├── kustomization.yaml
│   ├── pipeline.yaml
│   ├── tests.yaml
│   └── trivy.yaml
├── crds
│   ├── kustomization.yaml
│   └── standard-install.yaml
├── helm-releases
│   ├── istio-base.yaml
│   ├── istio-discovery.yaml
│   ├── istio-ingress-gateway.yaml
│   ├── kustomization.yaml
│   ├── nginx-gateway.yaml
│   └── sealed-secrets.yaml
├── helm-sources
│   ├── istio-namespace.yaml
│   ├── istio-repo.yaml
│   ├── kustomization.yaml
│   ├── nginx-gateway.yaml
│   └── sealed-secrets.yaml
├── k8s
│   ├── istio
│   │   ├── gateway.yaml
│   │   ├── kustomization.yaml
│   │   ├── namespace.yaml
│   │   ├── security.yaml
│   │   └── virtual-service.yaml
│   ├── llm
│   │   ├── deployment.yaml
│   │   ├── kustomization.yaml
│   │   ├── model-pvc.yaml
│   │   └── service.yaml
│   └── login
│       ├── deployment.yaml
│       ├── kustomization.yaml
│       └── service.yaml
├── my-cluster
│   ├── flux-system
│   │   ├── gotk-components.yaml
│   │   ├── gotk-sync.yaml
```

```
│   │   └── kustomization.yaml
│   └── kustomize-sources
│       ├── cdpipeline.yaml
│       ├── cipipeline.yaml
│       ├── crds.yaml
│       ├── helm-releases.yaml
│       ├── helm-sources.yaml
│       ├── istio.yaml
│       ├── llm.yaml
│       └── login.yaml
```

Cluster State

```
azureuser@qa-vm:~/cloud-native-handout-mil
NAME                                  READY
llmapp-deployment-7865d87cb-hkj2m     2/2
login-ddcdd89d5-8rf6v                 2/2
p5ci-b5mrx-build-push-django-pod      0/3
p5ci-b5mrx-build-push-llm-pod         0/3
p5ci-b5mrx-build-wheel-pod            0/2
p5ci-b5mrx-fetch-source-pod           0/2
p5ci-b5mrx-run-tests-pod              0/3
p5ci-b5mrx-scan-dep-django-pod        0/2
p5ci-b5mrx-scan-dep-llm-pod           0/2
p5ci-lbqzk-build-push-django-pod      0/3
p5ci-lbqzk-build-push-llm-pod         0/3
p5ci-lbqzk-build-wheel-pod            0/2
p5ci-lbqzk-fetch-source-pod           0/2
p5ci-lbqzk-run-tests-pod              0/3
p5ci-lbqzk-scan-dep-django-pod        0/2
p5ci-lbqzk-scan-dep-llm-pod           0/2

azureuser@qa-vm:~/cloud-native-handout-mil
NAME              TYPE           CLUSTER-II
kubernetes        ClusterIP      10.0.0.1
llmapp-service    LoadBalancer   10.0.193.8
loginservice      LoadBalancer   10.0.115.1

azureuser@qa-vm:~/cloud-native-handout-mil
NAME             REVISION                SU
cdpipeline       master@sha1:c7ecfed7    Fa
cipipeline       master@sha1:c7ecfed7    Fa
crds             master@sha1:c7ecfed7    Fa
flux-system      master@sha1:c7ecfed7    Fa
helm-releases    master@sha1:c7ecfed7    Fa
helm-sources     master@sha1:c7ecfed7    Fa
```

```
istio           master@sha1:c7ecfed7    Fa
llm             master@sha1:c7ecfed7    Fa
login           master@sha1:c7ecfed7    Fa
```

You can now submit your work using the `submitter -t task1`.

Reflection Questions

In your `answers/unit5answers.txt`, answer the following questions:

1. Why is it not advisable to manually tag the image versions in your deployment.yaml files
2. In this unit we used a HelmRepository and HelmRelease to install Istio. What is the difference between the two, and why do both need to exist?
3. Why does the order of resources in a kustomization.yaml matter? What could go wrong if deployment.yaml is listed before model-pvc.yaml in the LLM kustomization?

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform